

코드 검토 보고서

WorkLog 업무일지 관리 시스템

검토일	2026-05-27
대상 브랜치	main (커밋 c742704)
기술 스택	React 19.2 · TypeScript 5.8 · Vite 7 · Supabase · React Router 7
검토자	Claude Code (claude-sonnet-4-6)

1. 프로젝트 개요

KDN 내부용 업무일지 SPA. 프로젝트·인원 관리, 일지 CRUD, 진척률 추이 시각화, 주간·월간 요약 자동 생성 기능을 제공한다. Supabase(PostgreSQL)를 백엔드로 사용하며 GitHub Pages에 배포된다.

구분	내용
프론트엔드	React 19.2, TypeScript 5.8, Vite 7.3, React Router 7.13
백엔드/DB	Supabase (PostgreSQL) — @supabase/supabase-js 2.106
배포	GitHub Pages (CI/CD 워크플로우 포함)
라우트 수	10개 (Dashboard, Projects, Members, WorkLogs, Summary, 404)
주요 소스 파일	src/pages (9) · src/utills (2) · src/contexts (3) · src/components (2)

2. 아키텍처 평가 — 잘된 점

항목	내용
관심사 분리	db.ts (Supabase CRUD) ↔ storage.ts (순수 집계 로직) 분리가 명확하다
순수 함수 설계	dashboardKpi, progressSeries, summarize 등 비즈니스 로직이 외부 의존 없는 순수 함수로 작성되어 테스트 용이성이 높다
타입 안전성	도메인 타입(Project, Member, WorkLog)이 명확히 정의되고, DB row → 앱 타입 변환이 맵퍼 함수로 분리되어 있다
cascade 처리	deleteProject 는 FK cascade에 의존, deleteMember 는 project.member_ids 배열도 직접 정리한다
StrictMode 방어	모듈 레벨 _seeding 플래그로 React StrictMode의 이중 useEffect 호출을 방어한다 (db.ts:259)
진척률 단조 증가 보장	progressSeries 에서 날짜별 최댓값을 취하고 누적 max로 보정하여 진척률이 역행하지 않도록 설계했다 (storage.ts:62-67)

3. 버그 및 결함

3-1. useEffect 의존성 누락 — WorkLogEdit.tsx:42

버그

권한 검사에 사용하는 `currentMemberId`가 `useEffect` 의존성 배열에 빠져 있다. 페이지 진입 후 상단 사용자 드롭다운으로 현재 사용자를 바꾸면 권한 체크가 재실행되지 않아, 다른 사람의 일지에 접근한 상태에서 자신으로 전환해도 폼이 그대로 유지된다.

```
// 현재 코드 - currentMemberId가 누락됨
useEffect(() => {
  // log.memberId !== currentMemberId 체크 수행
  load().catch(console.error);
}, [id]); // ← currentMemberId 추가 필요
```

수정 방법

의존성 배열을 `[id, currentMemberId]`로 변경한다.

3-2. 로딩 중 KPI 값이 0으로 표시 — Dashboard.tsx

UX

데이터 로드 전 `KPI_INIT` (전부 0) 상태로 화면이 렌더링된다. 로딩 인디케이터가 없어 사용자가 실제 데이터가 없는 것으로 오인할 수 있다.

수정 방법

`loading` 상태를 추가하고, KPI 그리드에 스켈레톤 또는 스피너를 표시한다.

3-3. deleteMember 비원자적 업데이트 — db.ts:158-163

버그

프로젝트 멤버 배열 업데이트를 직렬 루프로 처리하므로, 중간에 오류가 발생하면 일부 프로젝트에서만 해당 멤버가 제거되고 나머지는 유지된다.

```
for (const p of affected ?? []) {
  await supabase.from('projects')
    .update({ member_ids: ... })
    .eq('id', p.id);
  // 오류 발생 시 이후 프로젝트는 업데이트 안 됨
}
```

수정 방법

`Promise.all()`로 일괄 처리하여 모든 업데이트를 병렬로 실행하고, 오류 시 전체 실패로 처리한다.

3-4. 미래 일자 일지 시 경과일 음수 표시 — Dashboard.tsx:227

UX

미래 날짜로 일지를 저장하면 `daysBetween()` 이 음수를 반환하여 화면에 `-1일 경과` 가 표시된다.

수정 방법

`Math.max(0, daysBetween(...))` 으로 음수를 방지한다.

4. 보안 취약점

4-1. [중요] 클라이언트 사이드 권한 검사

Critical

`currentMemberId` 는 `localStorage` 에 저장되므로 브라우저 콘솔에서 누구든 아래 한 줄로 임의 변경이 가능하다.

```
localStorage.setItem('wl_current_user', '다른사람의 -uuid');
```

이 경우 `WorkLogEdit.tsx:30` 의 권한 검사가 통과되어 타인의 일지를 수정·삭제할 수 있다.

`ProjectDetail.tsx:190` 의 수정/삭제 버튼 표시 제어도 동일하게 우회된다.

핵심 문제

Supabase Row Level Security(RLS)가 DB 레벨에서 시행되지 않는 한, 이 클라이언트 검사는 보안상의 의미가 없다. 현재 이 앱은 anon key로만 동작하여 모든 클라이언트가 동일한 DB 권한을 갖는다.

권고 사항

Supabase 대시보드에서 `work_logs` 테이블에 RLS 정책을 활성화하거나, 현재 내부 전용 도구임을 명확히 고지하고 신뢰 네트워크로 접근을 제한해야 한다.

4-2. 에러 무음 처리

경고

전체 페이지에서 비동기 오류가 `.catch(console.error)` 로만 처리된다. Supabase 연결 실패, 권한 오류 등이 콘솔에만 출력되고 사용자에게 안내되지 않는다.

권고

이미 구현된 `ToastContext` 의 `showToast('error', ...)` 를 활용한다.

5. 코드 품질 이슈

5-1. 폼 코드 중복 — WorkLogNew.tsx / WorkLogEdit.tsx

개선

두 파일의 폼 UI와 핸들러(items 추가/제거, preset 추가, 유효성 검사)가 거의 동일하다. `PRESETS` 상수도 양쪽에 복사되어 있다.

권고

공용 `WorkLogForm` 컴포넌트로 추출하면 유지보수성이 향상된다.

5-2. SVG 차트 코드 중복

개선

`Dashboard.tsx`의 `ProgressMiniChart`와 `ProjectDetail.tsx`의 인라인 SVG 차트가 동일한 path 계산 로직을 별도로 구현하고 있다. 공용 차트 컴포넌트로 통합을 권장한다.

5-3. any 타입 사용 — db.ts:7, 19, 28

개선

DB row 매퍼에서 `any` 타입과 `eslint-disable` 주석이 사용된다.

```
// eslint-disable-next-line @typescript-eslint/no-explicit-any
const toProject = (r: any): Project => ...
```

권고

Supabase CLI로 타입 자동 생성(`supabase gen types typescript`)하면 `any` 없이 완전한 타입 안전성을 확보할 수 있다.

5-4. window.confirm() 남용 — ProjectDetail.tsx:47, 53

UX

`confirm()`은 브라우저 기본 다이얼로그로 스타일 커스터마이징이 불가능하고 메인 스레드를 블록한다. 이미 `ToastContext`나 모달 패턴을 도입할 수 있는 구조이므로 커스텀 확인 다이얼로그 도입을 권장한다.

5-5. 전체 로그 일괄 로드

성능

`listLogs()`는 모든 `work_logs` 레코드를 한 번에 가져온다. Dashboard는 최근 6건과 KPI 집계만 필요함에도 전체를 로드하여 데이터가 수천 건으로 늘면 성능 저하 및 메모리 증가가 발생한다.

권고

Dashboard용 서버 사이드 집계 쿼리 분리 또는 최근 N건 제한 쿼리 도입을 검토한다.

6. 종합 평가

영역	점수	비고
아키텍처 설계	★★★★☆	db/storage 분리, 순수 함수 설계 우수
TypeScript 활용	★★★★☆☆	도메인 타입 양호, <code>any</code> 잔존
기능 완성도	★★★★☆	CRUD·요약·차트 전반적으로 동작
보안	★★☆☆☆	클라이언트 권한 검사 의존, RLS 미확인
코드 중복	★★★★☆☆	폼·차트 중복 존재
에러 처리	★★☆☆☆	대부분 <code>console.error</code> 로 무음 처리

7. 우선 조치 권고 사항

#	우선순위	항목	예상 공수
1	즉시 (보안)	Supabase RLS 정책 활성화 또는 내부망 접근 제한 명시	2-4h
2	단기	<code>WorkLogEdit</code> 의존성 배열에 <code>currentMemberId</code> 추가	0.5h
3	단기	<code>.catch(console.error)</code> → <code>ToastContext</code> 에러 toast로 교체	2h
4	중기	Dashboard 로딩 상태 추가 (스켈레톤 또는 스피너)	1h
5	중기	<code>deleteMember</code> → <code>Promise.all()</code> 병렬 처리로 원자성 개선	0.5h
6	중기	<code>WorkLogNew</code> / <code>WorkLogEdit</code> → 공용 <code>WorkLogForm</code> 컴포넌트 추출	3-4h
7	장기	Supabase CLI 타입 자동 생성으로 <code>any</code> 제거	2h
8	장기	<code>listLogs()</code> → 대시보드용 집계 쿼리 분리 및 페이지네이션 도입	4-6h

요약

항목 1-3번은 하루 이내 적용 가능한 수준입니다. 보안 이슈(4-1)가 가장 시급하며, 내부 전용 도구라 하더라도 RLS 또는 접근 제한 방침을 문서화해두는 것을 권장합니다.

